Group 19

CSE 4940

Dr. Bing Wang

Final Project Documentation

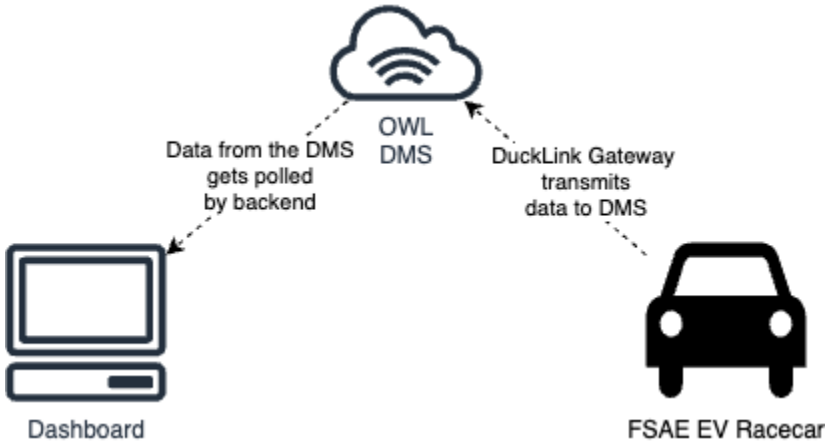
Project Description

The UConn Formula SAE team currently has no way to be able to view live data from their car while it is running. Their existing workflow requires driving the car, then stopping it and pulling data from the logger post-race. This data delay prevents the pit crew from being able to make real-time decisions about the vehicle's performance and safety. This limitation is impactful in a high-pressure racing environment where the detection of anomalies such as voltage instability of temperature changes can influence the race outcomes and vehicle safety. The goal of the dashboard component of this project is to solve this problem by building a live telemetry system that streams the car sensor data to a browser-based dashboard.

The MVP for the fall semester was a proof-of-concept dashboard that was built using Plotly Dash that could receive data over a WebSocket connection and be able to display it as a live graph. To simulate the real-world conditions, a mock DMS server was used which allowed the team to test the streaming and visualization. For the spring semester, the MVP was a fully functional, full-stack dashboard application. The frontend was built with React using the Recharts graph and charting library, and the backend was built using FastAPI, which interfaces with our Redis cache. The backend of the dashboard receives, parses, and stores data from the CAN bus in the Redis cache before being sent to the browser in real time via a WebSocket Connection. As far as the front end, we considered various approaches. First, it was with Dash by Plotly because it was relatively simple to set up and suited for this use case of data visualization. However, it combined both the frontend and backend into one application which became hard to maintain. Hence, for the spring semester, React and Recharts were chosen for the separation aspect as well as the UI flexibility. It was much easier to maintain long term and has a more scalable architecture.

In addition, the FSAE team uses an Orion Battery Management System which currently provides a fault flag indicating battery related faults with the car. This means that the team is only able to learn about the battery problems after they happen, leaving no time to make adjustments during races. Lithium-ion batteries, which power the FSAE vehicle, can be prone to failure which can make early detection crucial for safety. The goal for the machine learning component was to deploy an anomaly detection model to identify potential faults ahead of time and give the team time to respond. The models were trained using real operational data from FSAE racing, evaluated using the BMS fault flag as a weak label, and deployed on a Coral Dev Board.

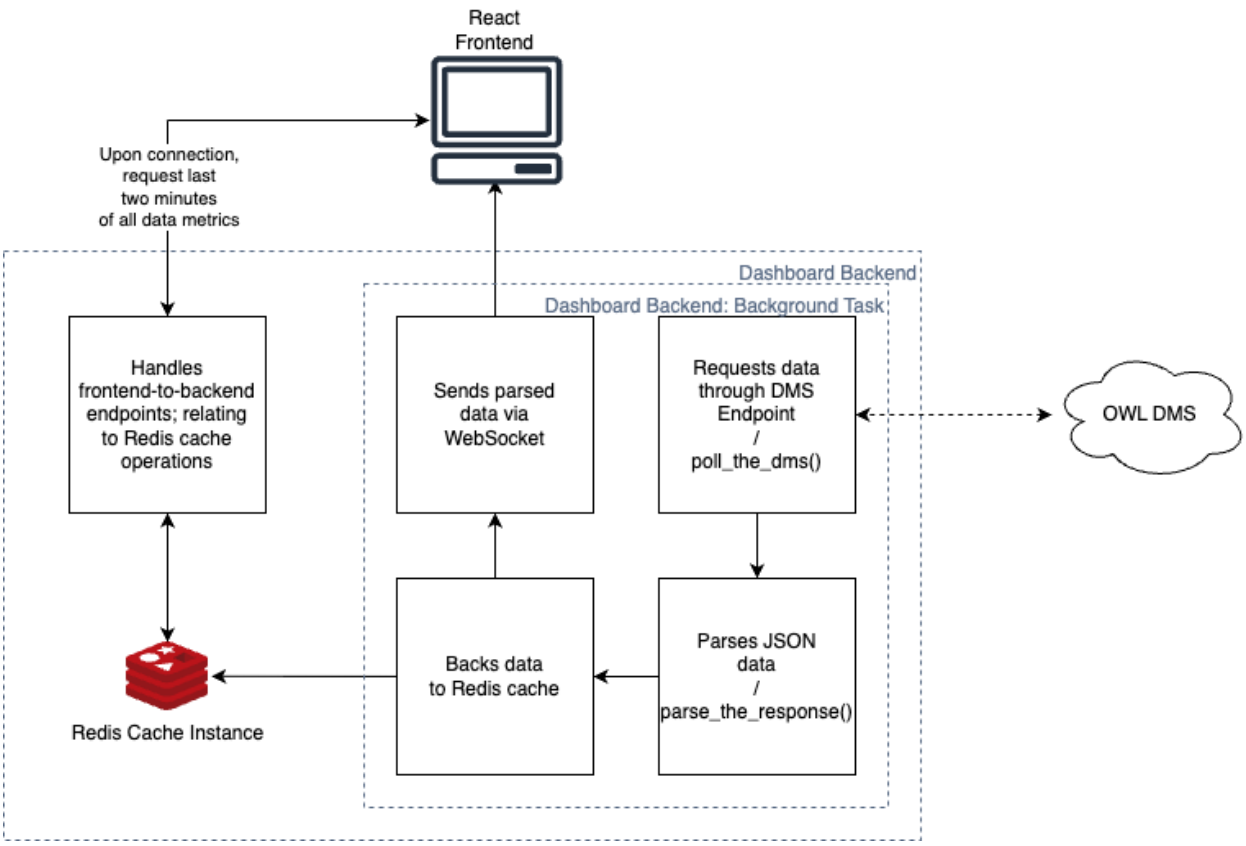
Design



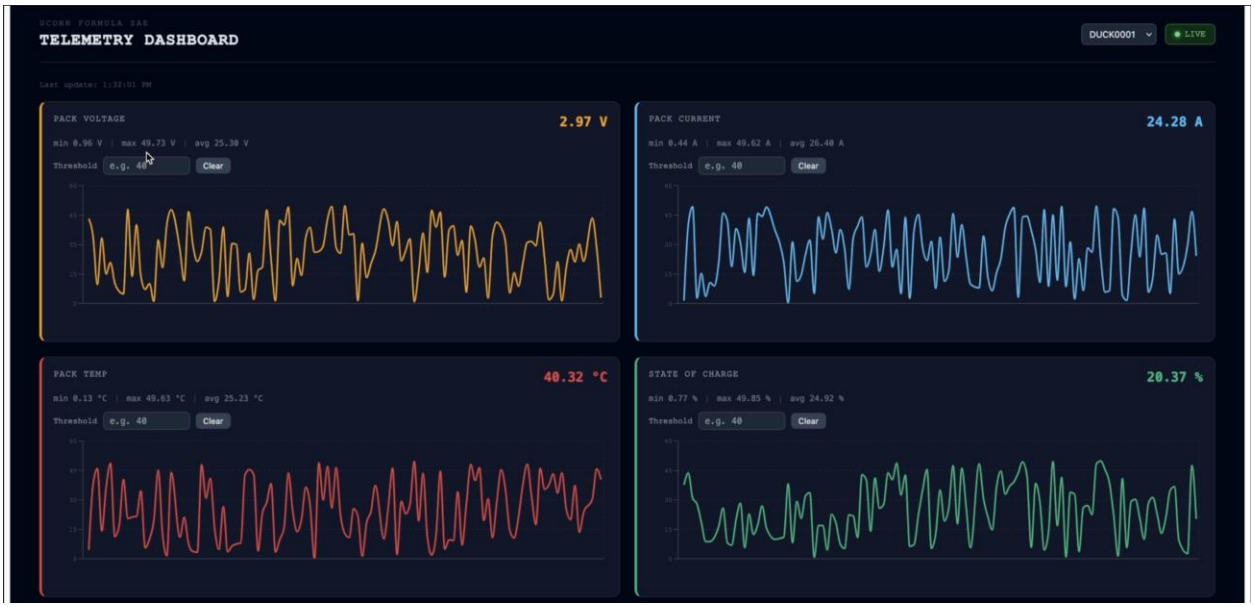
The diagram above shows what the project is at a high level. Essentially, sensors on the racecar collect and feed data into its internal CAN network. This data is read by a Duck Link Node and then transmitted via the LoRa mesh network, which is then forwarded to the OWL's Duck Management System (DMS) in the cloud. The dashboard would then pull the data from the DMS, parse it, and display it on respective graphs, for each data metric collected.

As per OWL's guidelines, we were required to use their Duck Links and the DMS for the project. With these restrictions pose software and hardware challenges regarding the design of this project, the respective flow, and by extension the dashboard. Since this project was under a joint team between the CSE and ECE departments, the ECE team largely handled the hardware, and we largely handled the software challenges. There were some factors that we considered handling these challenges, which include economic factors. We wanted to develop the dashboard in fashion that did not require expensive software or a subscription.

As mentioned before, the dashboard was initially developed using Dash by Plotly, a python framework tailored for data analysis and presentation. This framework coupled the frontend and backend logic together, which was hard to maintain. As the semester progressed, we transitioned to using a formal tech stack. The frontend was built with React and Recharts, which is a popular graph and charting library. The backend was built using FastAPI, a python server framework with WebSocket capabilities. In addition, the backend interfaces with an instance of Redis, a fast, in-memory key-value database. The diagram below shows how the backend operates.



The FastAPI backend is partitioned in three parts: Data acquisition and processing via an asynchronous background task, endpoints to handle various Redis operations, and WebSocket management. The background task is the centerpiece of the backend. Every second, the backend polls the DMS for the data as JSON packets, parses and stores the data in Redis. The data stored in the Redis cache are key-value pairs, where the key is the given metric of the electric racecar, which includes the car's voltage and amperage, and the value is a dictionary containing the value of this metric alongside a timestamp representing when it was collected. These key value pairs last exactly two minutes, meaning that the Redis cache only stores the last two minutes worth of collected data. After storing these key-value pairs, the backend notifies the WebSocket manager of new data, to which the manager promptly sends JSON data through the WebSocket. In addition, the backend maintains different endpoints, which handle basic get operations with the Redis cache. These operations include getting all data points of a given metric or getting all metrics that the dashboard is currently collecting. There is also an endpoint to list all the racecars the dashboard currently displays. However, since the Formula SAE team only has one EV racecar currently, the frontend only shows data for that racecar; it has the potential to show data for multiple active racecars.



The image above represents what the dashboard frontend looks. The dashboard contains graphs for all collected metrics, displaying the current value as well as the minimum, maximum, and average values for that metric. The user can also set a threshold to monitor data falling below or above a certain level. The React frontend is directly connected via WebSocket to receive live data. Upon initial browser connection, the frontend will get the last two minutes worth of data, for all given metrics. The frontend will begin listening to the WebSocket for incoming data thereafter.

For testing the dashboard, two mock servers were used. The first was a mock DMS server that generated random but realistic car sensor data. This test mimics the behavior of actually polling the DMS for the JSON data. When the team has access to the DMS, the team should be able to swap out the mock DMS URL to the actual DMS URL. The second was a mock CAN bus developed by the ECE team to simulate the incoming data sent over a serial bus.

Implementation

Our team learned the importance of organization early on in our project. Ensuring our team of four stayed on track in union with the ECE team and our project sponsor OWL Integrations required a consistent, defined development process. The development process we chose was Scrum, primarily because it allowed us to plan our weeks out in advance. Early in the semester our group met for sprint planning. During this meeting we first came up with the list of tasks we needed to complete throughout the course of the semester; we categorized them into two parent issues, the dashboard and machine learning components. After, we talked about which tasks we should complete each week. As an aside, we recommend that incoming students should be somewhat ambitious in their goals at this stage; you can always refine your backlog (unassigned, lists of tasks) overtime. Throughout the year, our group found the Jira board helpful

as a framework of what we should work on each week.

We would also like to emphasize the importance of scheduling meetings early. In the first weeks of the semester our group stayed proactive and reached out to our project sponsor, advisor, and the ECE team to schedule our weekly meetings; when2meet.com was extremely helpful here in finding a time that worked for everyone. We believe that scheduling meetings early will allow groups to coordinate early with their counterparts and ultimately allow more time for more significant project work. Specifically, we met every Tuesday and met with the ECE team and OWL Integrations every Friday. Later in the semester we started meeting with the ECE team to connect and test our system together. We found this schedule and timeline to work well for our needs and ultimately recommend something similar for new teams.

In GitHub, we have two separate repositories for the machine learning and dashboard components. We utilized feature branching for isolation of changes and agreed that this strategy worked well. Additionally, we hosted our cleaned datasets on Hugging Face, which was helpful in sharing with other team members and the ECE team.

Evaluation

The goal of this project was to develop a real-time display of an electric vehicle's data for the UConn Formula SAE team using OWL Integration's existing products. The devices they provided communicate with each other over the LoRa protocol, and the responsibility of the ECE team was to build a method of interface with the CAN network to collect the data from the existing sensors and send to another device not located on the vehicle. After meeting with the electrical and data acquisition leads on Formula SAE, the problem of battery anomaly detection was identified as an area where they could benefit from on the edge inference to improve their workflow. This system was built to increase the team's connectivity and safety when driving the electric vehicle.

Machine Learning Evaluation Metrics

Four different architectures were built as anomaly detection models including an Isolation Forest, an Isolation Forest with rolling mean, an Autoencoder, and an LSTM Autoencoder. These models were chosen based on research which identified them as frequently used solutions for anomaly detection problems [1], [2]. They were trained and evaluated on historical data provided from the Formula SAE's 2025 Electric Vehicle and its Battery Management System, BMS. The variable used as an indication of a fault was the BMS Disch Enable flag where a value of one indicated an anomaly. Evaluation of these models was based on three primary metrics: recall, false positive rate, and average lead time.

Recall is defined as the number identified anomalies divided by the actual number of anomalies in the dataset [3]. It measures the proportion of true faults that the model correctly indicates as anomalous. In the context of racing, a missed detection of a true anomaly presents a

higher risk than a false alarm. Failing to warn the team before an actual anomaly event means that the battery stops discharging current to the motor. This idea was confirmed by the Formula SAE team, and recall was retreated as the primary metric throughout the evaluation process.

The false positive rate calculates the percentage of incorrect positive predictions by dividing the number of normal driving points labeled anomalous by the total number of normal points [4]. This metric is also important to the evaluation of a model because a high false positive rate can lead a team to stop trusting the accuracy of the system and ignore all warnings of anomaly detection. A balance between the false positive rate and recall is necessary in order to develop a model which is both useful and correct for the Formula SAE team's goals.

One of the main indications that a model is performed as expected is the measure of how far in advance of the actual BMS faults an anomaly is detected. A positive average lead time means that the model has flagged an anomaly event prior to the BMS, and it is predicting earlier that the current threshold checks were implemented. To evaluate this, a lookback window was used to identify when the model is flagging a particular anomaly event. However, due to the high recall and false positive rates that are explained in the Model Performance Results section, this metric may not be fully representative of a model's ability to identify an actual anomaly event in advance but rather the model's tendency to overclassify datapoints as anomalous.

The last metric used to directly compare the different models' performance was inference latency. Inference latency is defined to be the time between when a machine learning model receives an input and when it produces a prediction [5]. As the model will be running on the edge, this metric is an important consideration when determining overall applicability and performance of a model.

Model Performance Results

The models were all trained on one session of data which contained 37,180 rows of data over the course of thirty minutes. This session was determined to be the most ideal for training due to its lack of faults and consistent driving statistics, which are explained in more detail in the Dataset Evaluation Section. The following table shows the results from evaluating a session with 255 fault rows and 10,902 normal rows. The model is credited with early detection if within a fifteen second window prior to the actual anomaly event it flagged a datapoint as anomalous.

	Isolation Forest	IF with rolling mean	Autoencoder	LSTM Autoencoder
Recall	96.86%	99.61%	1.57%	17.25%
False Positive Rate	76.98%	66.46%	23.81%	75.15%
Lead Time	15.000s	14.500s	15.000s	15.000s

Inference Latency mean	12.39ms	15.86ms	76.59ms	76.145ms

The model that achieved the highest recall was the Isolation Forest with rolling mean with a value of 99.61%. It correctly flagged 254 datapoints as anomalies out of the 255 recorded instances. This means that the model was able to detect a pattern between the faults; however, the false positive rate of 66.46% means that the model was over classifying what it considered to be an anomaly. An important caveat is that a portion of these datapoints included in the false positive rate calculation are the model labeling an anomaly in advance. While they are technically normal data that is flagged as anomalous, this is what the model is meant to be doing. Further investigation into the false positives showed that not all incorrectly identified anomalous can be explained in this manner. There are instances of the model labeling a datapoint as positive, far in advance of any fault. These cases are representative of the differences in the training and evaluation conditions which is expanded in the Dataset Evaluation section.

The deep learning models did not have the same recall of success that the Isolation Forest models showed. The Autoencoder had a false positive rate of 23.81% but with correctly identifying only 1.57% of the actual anomalies, meaning it was too conservative in its classifications. The LSTM version of the autoencoder had an improved recall of 17.25% but a considerable increase in its false positive rate. The average inference of latency for the autoencoder models was significantly longer than the Isolation Forest models, which is an important factor to account for when deploying a model on the edge.

These performances indicate an underlying problem in the quality of data. The two main architectures detect anomalies using different fundamental structures. The Isolation Forest is built on the idea that anomalies are separate from the rest of the data points and can be isolated from the rest [6]. An autoencoder, on the other hand, works to reconstruct data from a more compressed reconstruction and the error when reconstructing is much larger for an anomalous datapoint [7]. The training data and test data come from different operating conditions, which is true for any two instances of driving due to the variation in conditions, driver, and focus of the session. The models cannot identify the difference between new driving conditions and an actual precursor of a fault.

Hyperparameter Tuning

Given that the Isolation Forest with rolling mean model demonstrated the highest recall and most overall promising performance, further hyperparameter tuning was experimented with to determine if performance could be improved. The contamination rate, threshold quantile, and rolling window size were altered to try to find the right combination of these variables. The

overall results showed that no combination would produce both a high recall value and low false positive rate, reinforcing the theory that the limiting factor is the data rather than the model itself.

The first variable tested was the contamination parameter which defines the expected proportion of anomalies so the algorithm can set its decision boundary for prediction [6]. It is used during the fitting of the model in the training stage. The contamination value should be in the range from 0 to 0.5. Values of 0.01, 0.05, and 0.1 were tested. These values were chosen to evaluate what a shift in this value resulted in. As shown in the table below, these three contamination values produced the same recall and false positive rate. The differences do not alter the results because our training data does not have anomalies, so this threshold is not used during prediction.

Contamination	Recall	False Positive Rate
0.01	99.61%	66.46%
0.05	99.61%	66.46%
0.10	99.61%	66.46%

The threshold quantile value is used during inference to determine what percentage of the top anomaly scores are alerted. This value does not use the contamination value from scikit-learn but rather computes a threshold by determining the test points with an anomaly score above this quantile. A lower quantile value will be more apt at correctly identifying anomalies but also increase the false positive rate. The following table shows the tests done to visualize the effects of the different quantile values on the Isolation Forest rolling mean model's performance. This relationship is a direct tradeoff between higher recall and higher false positive rates. Therefore, in attempting to lower the false positive rates through this hyperparameter, the model will also not be able to detect all of the actual identified anomalies in the dataset.

Threshold Quantile	Recall	False Positive Rate
0.70	100.00%	99.03%
0.80	100.00%	96.48%
0.90	100.00%	82.80%
0.95	100.00%	75.06%
0.96	100.00%	71.19%
0.97	99.61%	66.46%
0.98	80.78%	58.15%
0.99	62.75%	49.99%

The difference between the Isolation Forest model with and without the rolling mean is the addition of statistics which aim to incorporate timeseries data rather than having the model interpret the different points individually. The rolling window size variable controls how large a time range is averaged together to compare to the current data point. Different window sizes were tested to evaluate how this would impact the performance metrics of the model. These were done at a fixed quantile value of 0.97 as this was the best performing parameter from previous tests. The results show that as the window size increases, both the recall and false positive rates decrease. The model becomes less sensitive due to a smoothing out effect of specific datapoints which might differ in value from those around it. Setting the default window size to be 25 rows or 1.25s was confirmed to be the correct choice for this model based on these results.

Window Size	Duration	Recall	False Positive Rate
15 rows	0.75s	100.00%	71.27%
25 rows	1.25s	99.61%	66.46%
35 rows	1.75s	96.86%	62.30%
50 rows	2.50s	91.76%	58.46%

Dataset Evaluation

Initially, the training and testing of the models was completed with all twenty-seven sessions of data provided from the 2025 Formula SAE Electric Vehicle. However, further investigation revealed that the specific dataset used significantly impacted the results of the model. Originally, the models were trained on eighteen datasets which contained no faulty data and tested on the nine files which contained anomaly events. The results did not represent the idea that the model was running successfully. The sessions were crossed referenced with a database of notes taken by the drivers and engineers to identify more information about the conditions and what caused the faults to occur. The following table provides additional insight into how variable the files are.

File	Total Rows	Fault Rows	Avg RPM	Max Temp	Min SOC
2025_06_14-01	1,439	0	0	19°C	80%
2025_06_14-02	1,459	0	0	19°C	80%
2025_06_14-03	23,591	0	1,334	32°C	60%
2025_06_14-04	2,379	0	0	29°C	60%
2025_06_21-01	36,347	0	2,147	38°C	60%

2025_07_12-01	6,975	451	1,499	55°C	40%
2025_07_12-02	6,773	0	1,804	46°C	26%
2025_07_12-03	10,917	0	1,621	47°C	80%
2025_07_12-04	2,019	0	2,006	48°C	64%
2025_07_12-05	2,681	19	963	50°C	60%
2025_07_12-06	2,300	58	1,075	50°C	60%
2025_07_12-07	19,019	0	0	46°C	60%
2025_07_12-08	1,555	220	2,322	51°C	60%
2025_07_12-09	2,349	356	489	40°C	94%
2025_07_12-10	1,301	0	957	41°C	94%
2025_07_26-01	6,641	0	432	34°C	87%
2025_07_26-02	17,899	0	0	32°C	87%
2025_07_26-03	5,902	0	1,860	45°C	80%
2025_07_26-04	11,181	255	2,272	55°C	40%
2025_07_26-05	3,779	1,691	0	25°C	50%
2025_07_26-06	1,659	0	0	25°C	50%
2025_07_26-07	1,159	1,159	0	0°C	0%
2025_08_01-01	4,196	0	1,636	28°C	80%
2025_08_01-02	1,653	0	1,836	33°C	80%
2025_08_01-03	2,686	411	964	30°C	80%
2025_08_01-04	759	0	0	29°C	50%
2025_08_01-05	2,499	0	0	29°C	50%

Based on this insight, ten of the files were removed from consideration due to the car being stationary for the entire duration of the session. Additionally, this exploration revealed just how different the conditions are between different sessions. The table above highlights some of these shifts within the average RPM, max temperature, and minimum state of charge during the session. The training data would provide the model with a representation of what is considered normal and then during evaluation the model is unable to differentiate between anomalous data and different normal data from the learned session.

The best identified dataset for training was the session 2025_06_21-01 which was an endurance test from the Michigan competition. This dataset consisted of 36,347 rows and zero faults. The test file was 2025_07_26-04, which was an independent test session conducted by the team since none of the competition sessions had fault instances. This file contains 11,181 rows and 255 fault points. This single train and test pair was used due to its average variable values throughout. Training and testing on a variety of different sessions demonstrated that the model is not able to generalize new conditions that it was not trained on. This presents a challenge as there are many factors that make each driving session unique for the vehicle, including the track, driver, purpose of session, and condition of the car. The dataset is also limited due to the number of faults present across the dataset. There are seven different fault events that make up approximately six percent of the total points. This imbalance adds to the difficulty of generalization that the model is experiencing.

Dashboard Evaluation Metrics

The dashboard was built for UConn's Formula SAE team; hence their feedback was one of the main methods of evaluation used throughout the year. Meetings to gauge their needs of what a live display of data should consist of, and demonstrations of progress occurred regularly. Pack voltage, pack current, pack temperature, state of charge, minimum cell voltage, and BMS LV input were the variables they wanted to see live graphs of when the car was driving. They reviewed the initial design and provided feedback. This was implemented to tailor the system directly to their needs. They indicated which datapoints were most important to represent, what the layout of the dashboard should include, and how it could be accessed. This evaluation process was utilized to provide the exact specifications that the team wanted rather than designing it all prior to conversing with the clients.

While the usefulness of the dashboard was evaluated through the Formula SAE team's feedback, the functionality of the system was tested in two main ways. OWL Integrations has a cloud platform called the Duck Management System, DMS, which they wanted the project to incorporate. To meet these needs, the dashboard was designed to interface with this database through a WebSocket connection. This product was not fully compatible with the entire project as the devices provided from OWL did not actually communicate with the DMS, so a mock server was implemented instead. This simulated how the dashboard would interact with a database through pulling and updating the graphs based on the received information. The Mock DMS was a small FastAPI server with one endpoint that was called every second to get random data. These integration tests were designed to confirm functionality of the dashboard with external systems like the DMS or other mock database servers.

The other form of integration testing of the dashboard occurred when combining the systems built by the ECE half of the project. The devices they were provided are called T-beams. Their setup included the sending of previous Formula SAE data in a CAN format from a Raspberry Pi to simulate how it would be on the vehicle, capturing this data on one T-beam, and

then sending it over LoRa to another T-beam. Since the T-beams could not access the DMS, the dashboard was modified to receive data over a serial connection and display the incoming information from the receiving T-beam. While this was not the intended format of the project, the design was adapted to work with the constraints of the existing devices.

Team Evaluation and Development Process

The team evaluated itself in a variety of different ways throughout the course of the year. The most important metric used was how much learning occurred. Many mistakes were made and pivots in approaches, but each step was taken as an opportunity to grow. Understanding why things were performing as they were or why certain implementations were needed was more important to the team than the overall result. This project had the freedom to try new areas of computer science that all team members did not have previous experience with. Rather than be concerned about the lack of knowledge, the team embraced this as the opportunity it was.

The project was successful because it completed the goal of providing live data for the Formula SAE team while they were driving. A machine learning model was built to compute battery anomaly detection on the edge. This system provides the team with significantly more capabilities than they had previously. The success was confirmed with their enthusiasm for implementing this system going forward. While the actual performance metrics of the anomaly detection model were not completely successful in their generalization abilities, it does inform the team in advance of when a battery anomaly event occurs. The dashboard graphs the metrics requested by the team over the course of a driving session. The evaluation process and overall results indicate that the project was successful.

The team used Git to manage the code between the four members. Pull requests were used to help with management of the code base. Branches were created off the main one to ensure compatibility of proposed changes before merging. This also provided the opportunity to document what the new code was doing, why these changes were necessary, and what impact this would have on the existing system. Using pull requests allows for greater organization and collaboration between team members.

Unit testing and integration testing were performed throughout the entire project to ensure functionality and correctness of both the machine learning models and the dashboard. While formal test files were not implemented for either component, testing was conducted through investigations, experimentation, and verification of outputs throughout development. For machine learning, different components were run in isolation to verify correct behavior. The system was broken into the loading of the data, the training of the model, and the evaluation of the model. The dashboard verified that the incoming data was accurately represented in the graphs. Integration testing mainly occurred when connecting with what the ECE team developed.

Changes were necessary to mesh the two teams' work, but the entire system was set up and run which confirmed the performance of the project.

New Knowledge

The machine learning team (Maeve & Jobe) did not have any previous machine learning knowledge, so a significant portion of the project required continuous learning over the course of the year. The brainstorming and initial drafting of the project required research and basic tutorials about machine learning models and their implementations. There was a variety of resources used including YouTube videos, blog posts, and research papers to understand how to build an anomaly detection method. The PyTorch website and Scikit-learn documentations pages were also very useful when implementing. Having no prior experience in this area of computer science allowed for a unique opportunity to completely learn something new outside of the classroom. Tutorials provided the fundamental understanding of different aspects of machine learning algorithms, and research papers helped visualize different applications throughout the learning process. Additionally, Taraquar and Owen from OWL Integrations were a great resource for their machine learning expertise. Overall, we would recommend this combined format to a new audience with no or limited experience. The initial research, implementation of different methods, and experimentation with the results would take a new group around three to four months.

The dashboard team came into this project with minimal experience in web development and little to less experience in building a full stack system. A big portion of this project was learning new technologies through the course of the year. Understanding how to implement a WebSocket was a critical piece of the system. We learned about FastAPI through the official documentation at fastapi.com. We worked on getting comfortable with the framework and worked on using Redis through tutorials. On the frontend, React was a somewhat familiar tool to built the updating component and Recharts was also completely new and learned through the official documentation at https://recharts.github.io/. For an incoming team, we recommend learning in the order of FastAPI, WebSocket and then Redis following React with Recharts.

	Project Description	Design	Implementation	Evaluation	New Knowledge
Jobe					
Maeve					
Esha					

Vlad					

Citations

[1] A. A. Neloy and M. Turgeon, “A comprehensive study of auto-encoders for anomaly detection: Efficiency and trade-offs,” *Machine Learning with Applications*, vol. 17, p. 100572, Sep. 2024, doi: https://doi.org/10.1016/j.mlwa.2024.100572.

[2] C. Stanton, G. Katz, and D. Song, “Isolation Forest for Anomaly Detection,” Center for Energy Efficient Electronics Science, Univ. of California, Berkeley, 2015. [Online]. Available: https://e3s-center.berkeley.edu/wp-content/uploads/2017/08/RET_CStanton-2015.pdf.

[3] K. M. Ting, “Precision and Recall,” in *Encyclopedia of Machine Learning*, C. Sammut and G. I. Webb, Eds. Boston, MA, USA: Springer, 2011, doi: https://doi.org/10.1007/978-0-387-30164-8_652.

[4] GeeksforGeeks, “False positive rate,” *GeeksforGeeks*, Jul. 23, 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/false-positive-rate/.

[5] Ultralytics, “Inference latency,” *Ultralytics Glossary*. [Online]. Available: https://www.ultralytics.com/glossary/inference-latency.

[6] Scikit-learn developers, “IsolationForest,” *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html.

[7] D. Bergmann and C. Stryker, “Autoencoder,” *IBM Think*. [Online]. Available: https://www.ibm.com/think/topics/autoencoder.